Using RObhdsiWebApi

Gowtham Rao

2026-10-12

Contents

1 Introduction **1**

2 WebApi configurations and RObhdsiWebApi **1**
2.1 WebApi Analytical categories. 2

3 Framework of RObhdsiWebApi **2**
3.1 Naming conventions of RObhdsiWebApi 2

4 Concept Set **5**
4.1 if want to print ready expression of the concept set definition 7

5 Applications of RObhdsiWebAPi **7**
5.1 Cohorts/Characterization/Incidence rate 7
5.2 Characterization 8

RObhdsiWebApi is part of HADES.

1 Introduction

From Package Readme

> RObhdsiWebApi is a R based interface to 'WebApi' (OHDSI RESTful services), and performs GET/PULL/POST/DELETE calls via the WebApi. All objects starting from R or output to R - are analysis ready R-objects like list and data.frame. The package handles the intermediary steps by converting R-objects to JSON and vice versa. To ensure r-objects are analysis ready, the objects are type converted where possible, e.g. date/date time are converted from string to POSIXct.
>
> This package makes reproducible research easier, by offering ability to retrieve detailed study specifications, transport study specifications from one instance to another, programmatically invoke the generation of a sequence of steps that are part of a study, manage running studies in batch mode.

This document will attempt to explain how RObhdsiWebApi maybe used to achieve reproducible research.

2 WebApi configurations and RObhdsiWebApi

To successfully use RObhdsiWebApi, it is necessary to have an active 'WebApi' endpoint with a known baseUrl such as "http://server.org:80/WebAPI". 'WebApi' has many functional categories.

To ensure reproducibility of work it is best to know the version of the WebApi (i.e. Atlas backend) being used. An easy way to do that is (and output maybe included in your study results)

```
version <- ROhdsiWebApi:::getWebApiVersion(baseUrl = baseUrl)
message1 <- paste0('This Vignette was created using WebApi version: ',
                   version,
                   ' on baseUrl: ',
                   baseUrl,
                   ". The CDM had the following source data configured: ")
cdmSources <- ROhdsiWebApi::getCdmSources(baseUrl = baseUrl)
priorityVocabulary <- ROhdsiWebApi::getPriorityVocabularyKey(baseUrl = baseUrl)
```

The object `version` will show your webApi version. Example: .

```
cdmSources
```

The priority vocabulary for the WebApi is .

We can also perform checks on the WebApi, example - we may want to see if the 'HCUP' & 'SYNPUF1K' is a valid SourceKey in the current webApi.

```
ROhdsiWebApi::isValidSourceKey(sourceKeys = c('HCUP', 'SYNPUF1K'), baseUrl = baseUrl)
```

2.1 WebApi Analytical categories.

WebApi maybe considered to have certain modular analytic categories. RohdsiWebApi supports the following categories:

```
#> Warning: Use of .data in tidyselect expressions was deprecated in tidyselect 1.2.0.
#> i Please use `"categoryFirstUpper"` instead of `.data$categoryFirstUpper`
#> This warning is displayed once per session.
#> Call `lifecycle::last_lifecycle_warnings()` to see where this warning was generated.
```

Category	Features
ConceptSet	Functions for interfacing with ConceptSet in WebApi
Cohort	Functions for interfacing with Cohort in WebApi
IncidenceRate	Functions for interfacing with IncidenceRate in WebApi
Characterization	Functions for interfacing with Characterization in WebApi
Pathway	Functions for interfacing with Pathway in WebApi

3 Framework of RohdsiWebApi

RohdsiWebApi maybe better understood by having atleast a high level understanding of CRUD framework for WebApi, i.e. the GET, PUT, DELETE, POST calls to the API. See the documentation of the WebApi.

For each supported category, RohdsiWebAPi performs GET, PUT, DELETE, POST calls to WebApi in background. The details of what calls are actually performed is less important to an analyst, but it is useful to understand the naming conventions of RohdsiWebApi.

3.1 Naming conventions of RohdsiWebApi

Most functions in RohdsiWebApi start with an action oriented 'verb' - such as

Function Name	Description
authorizeWebApi	Authorize Web Api
cancelCharacterizationGeneration	Cancel Characterization Generation

Function Name	Description
cancelCohortGeneration	Cancel Cohort Generation
cancelGeneration	Cancel Generation
cancelIncidenceRateGeneration	Cancel Incidence Rate Generation
cancelPathwayGeneration	Cancel Pathway Generation
checkInputFileEncoding	Check Input File Encoding
convertConceptSetDefinitionToTable	Convert Concept Set Definition To Table
createApiKey	Create Api Key
createConceptSetWorkbook	Create Concept Set Workbook
deleteCharacterizationDefinition	Delete Characterization Definition
deleteCohortDefinition	Delete Cohort Definition
deleteConceptSetDefinition	Delete Concept Set Definition
deleteDefinition	Delete Definition
deleteIncidenceRateDefinition	Delete Incidence Rate Definition
deletePathwayDefinition	Delete Pathway Definition
detectCharacterizationsByName	Detect Characterizations By Name
detectCohortsByName	Detect Cohorts By Name
detectConceptSetsByName	Detect Concept Sets By Name
detectIncidenceRatesByName	Detect Incidence Rates By Name
detectPathwaysByName	Detect Pathways By Name
existsCharacterizationName	Exists Characterization Name
existsCohortName	Exists Cohort Name
existsConceptSetName	Exists Concept Set Name
existsIncidenceRateName	Exists Incidence Rate Name
existsPathwayName	Exists Pathway Name
exportCohortDefinitionSet	Export Cohort Definition Set
getCdmsources	Get Cdmsources
getCharacterizationDefinition	Get Characterization Definition
getCharacterizationDefinitionsMetadata	Get Characterization Definitions Metadata
getCharacterizationGenerationinformation	Get Characterization Generationinformation
getCharacterizationResults	Get Characterization Results
getCohortDefinition	Get Cohort Definition
getCohortDefinitionExpression	Get Cohort Definition Expression
getCohortDefinitionName	Get Cohort Definition Name
getCohortDefinitionSql	Get Cohort Definition Sql
getCohortDefinitionsMetadata	Get Cohort Definitions Metadata
getCohortGenerationinformation	Get Cohort Generationinformation
getCohortInclusionrulesandcounts	Get Cohort Inclusionrulesandcounts
getCohortResults	Get Cohort Results
getCohortSql	Get Cohort Sql
getConceptSetDefinition	Get Concept Set Definition
getConceptSetDefinitionBySourceKey	Get Concept Set Definition By Source Key
getConceptSetDefinitionsMetadata	Get Concept Set Definitions Metadata
getConcepts	Get Concepts
getDefinition	Get Definition
getDefinitionsMetadata	Get Definitions Metadata
getGenerationinformation	Get Generationinformation
getIncidenceRateDefinition	Get Incidence Rate Definition
getIncidenceRateDefinitionsMetadata	Get Incidence Rate Definitions Metadata
getIncidenceRateGenerationinformation	Get Incidence Rate Generationinformation
getIncidenceRateResults	Get Incidence Rate Results
getMyPermissions	Get My Permissions
getPathwayDefinition	Get Pathway Definition

Function Name	Description
getPathwayDefinitionsMetadata	Get Pathway Definitions Metadata
getPathwayGenerationinformation	Get Pathway Generationinformation
getPathwayResults	Get Pathway Results
getPersonProfile	Get Person Profile
getPriorityvocabularykey	Get Priorityvocabularykey
getResults	Get Results
getSourceconcepts	Get Sourceconcepts
getUserInformation	Get User Information
getUsersFromRole	Get Users From Role
getWebApiVersion	Get Web Api Version
insertCohortDefinitionInPackage	Insert Cohort Definition In Package
insertCohortDefinitionSetInPackage	Insert Cohort Definition Set In Package
invokeCharacterizationGeneration	Invoke Characterization Generation
invokeCohortGeneration	Invoke Cohort Generation
invokeGeneration	Invoke Generation
invokeIncidenceRateGeneration	Invoke Incidence Rate Generation
invokePathwayGeneration	Invoke Pathway Generation
isvalidCharacterizationId	Isvalid Characterization Id
isvalidCohortId	Isvalid Cohort Id
isvalidConceptSetId	Isvalid Concept Set Id
isvalidId	Isvalid Id
isvalidIncidenceRateId	Isvalid Incidence Rate Id
isvalidPathwayId	Isvalid Pathway Id
isvalidSourceKey	Isvalid Source Key
postCharacterizationDefinition	Post Characterization Definition
postCohortDefinition	Post Cohort Definition
postConceptSetDefinition	Post Concept Set Definition
postDefinition	Post Definition
postIncidenceRateDefinition	Post Incidence Rate Definition
postPathwayDefinition	Post Pathway Definition
resolveConceptSet	Resolve Concept Set
securityEnabled	Security Enabled
setAuthHeader	Set Auth Header
setCohortPermission	Set Cohort Permission
updateCohortDefinition	Update Cohort Definition
updateConceptSetDefinition	Update Concept Set Definition
updateDefinition	Update Definition
whoAmI	Who Am I

Most of the functions start with the following verbs:

Function Verb	Number Of Functions
Get	37
Isvalid	7
Delete	6
Post	6
Cancel	5
Detect	5
Exists	5
Invoke	5
Update	3

Function Verb	Number Of Functions
Create	2
Insert	2
Set	2
Authorize	1
Check	1
Convert	1
Export	1
Resolve	1
Security	1
Who	1

A function to get `Definition` is `getDefinitionMetaData` function. This is a general function that is able to get the Metadata for all specifications within a category.

```
ROhdsiWebApi::getDefinitionsMetadata(baseUrl = baseUrl,
                                     category = 'cohort') %>%
  arrange(.data$id) %>%
  rename_all(.funs = SqlRender::camelCaseToTitleCase) %>%
  head()
```

The same output may be achieved using

```
ROhdsiWebApi::getCohortDefinitionsMetaData(baseUrl = baseUrl) %>%
  arrange(.data$id) %>%
  rename_all(.funs = SqlRender::camelCaseToTitleCase) %>%
  head()
```

This is a generic framework that applies to most WebApi categories, and supports different types of CRUD functionalities like deleteConceptSetDefinition() vs deleteDefinition(category = 'conceptSet').

4 Concept Set

Please review 'Concept sets - The Book of OHDSI'

We commonly post concept set expression into WebApi/Atlas, or try get an expression from Atlas/WebApi based on a conceptSetDefinitionId.

Example: lets say we have concept set expression as follows, that is being used for a Rheumatoid Arthritis study.

```
jsonExpression <- '{
  "items": [
    {
      "concept": {
        "CONCEPT_ID": 81097,
        "CONCEPT_NAME": "Feltys syndrome",
        "STANDARD_CONCEPT": "S",
        "STANDARD_CONCEPT_CAPTION": "Standard",
        "INVALID_REASON": "V",
        "INVALID_REASON_CAPTION": "Valid",
        "CONCEPT_CODE": "57160007",
        "DOMAIN_ID": "Condition",
        "VOCABULARY_ID": "SNOMED",
```

```
        "CONCEPT_CLASS_ID": "Clinical Finding"
      },
      "isExcluded": true,
      "includeDescendants": false,
      "includeMapped": false
    },
    {
      "concept": {
        "CONCEPT_ID": 80809,
        "CONCEPT_NAME": "Rheumatoid arthritis",
        "STANDARD_CONCEPT": "S",
        "STANDARD_CONCEPT_CAPTION": "Standard",
        "INVALID_REASON": "V",
        "INVALID_REASON_CAPTION": "Valid",
        "CONCEPT_CODE": "69896004",
        "DOMAIN_ID": "Condition",
        "VOCABULARY_ID": "SNOMED",
        "CONCEPT_CLASS_ID": "Clinical Finding"
      },
      "isExcluded": false,
      "includeDescendants": true,
      "includeMapped": false
    },
    {
      "concept": {
        "CONCEPT_ID": 4035611,
        "CONCEPT_NAME": "Seropositive rheumatoid arthritis",
        "STANDARD_CONCEPT": "S",
        "STANDARD_CONCEPT_CAPTION": "Standard",
        "INVALID_REASON": "V",
        "INVALID_REASON_CAPTION": "Valid",
        "CONCEPT_CODE": "239791005",
        "DOMAIN_ID": "Condition",
        "VOCABULARY_ID": "SNOMED",
        "CONCEPT_CLASS_ID": "Clinical Finding"
      },
      "isExcluded": false,
      "includeDescendants": true,
      "includeMapped": false
    }
  ]
}'
```

Lets call this concept set expression - '[ROhdsiWebApi Vignette] Rheumatoid Arthritis concept set'.

We will need to check if there is a concept set by this name.

```
# check if there is a concept set by this name, if yes, delete it
exists <- ROhdsiWebApi::existsConceptSetName(conceptSetName = conceptSetName, baseUrl = baseUrl)
exists
```

If there is a concept set with this name, then we could either choose another name - or delete the old concept set. For this vignette we have chosen to delete any matching concept set as follows:

```
if (!isFALSE(exists)) {
  ROhdsiWebApi::deleteConceptSetDefinition(conceptSetId = exists$id, baseUrl = baseUrl)
```

```
}
```

Now we need to ensure the JSON expression above is converted to R-data object. Note: By design, ROhdsiWebAPi does not accept JSON. It needs to be converted to R (list) expression

We can now post this R-object into WebApi as follows:

```
returnFromPostRequest <- ROhdsiWebApi::postConceptSetDefinition(baseUrl = baseUrl,
                                                                conceptSetDefinition = rExpression,
                                                                name = conceptSetName)
```

If successful, we will get a return object as follows into R.

The id of the newly posted concept-set definition is `returnFromPostRequest$id`. We can now use this concept-set for many concept set queries eg.,

4.1 if want to print ready expression of the concept set definition

`createConceptSetWorkbook` maybe used to create an Excel workbook of the concept set.

If we want a list of all conceptId's (including descendants) from the concept set expression

The concept set expression json expression can be recaptured from WebApi as follows

Similar framework maybe used with other WebApi categories such as Cohorts/Characterization/Incidence Rate.

5 Applications of ROhdsiWebAPi

A valuable feature of ROhdsiWebApi is that it is able to get full result set into R, as a data frame object. Results of Cohort, Characterization, Incidence Rate, Pathway maybe obtained. This data frame may then be converted to publication ready material by using packages like Officer, flextable. The functions in ROhdsiWebAPi maybe used to create dynamic R-shiny apps that allow user to interact with WebApi and select cohort definitions, concept sets for review or modifications. ROhdsiWebApi may be used to build 'mini versions' of Atlas that is project specific - by directly interacting with WebApi using R.

5.1 Cohorts/Characterization/Incidence rate

Please review 'What is a cohort - The Book of OHDSI'.

> We define a cohort as a set of persons who satisfy one or more inclusion criteria for a duration of time. The term cohort is often interchanged with the term phenotype. Cohorts are used throughout OHDSI analytical tools and network studies as the primary building blocks for executing a research question.
>
> A cohort is defined as the set of persons satisfying one or more inclusion criteria for a duration of time. One person may qualify for one cohort multiple times during non-overlapping time intervals. Cohorts are constructed in ATLAS by specifying cohort entry criteria and cohort exit criteria. Cohort entry criteria involve selecting one or more initial events, which determine the start date for cohort entry, and optionally specifying additional inclusion criteria which filter to the qualifying events. Cohort exit criteria are applied to each cohort entry record to determine the end date when the person's episode no longer qualifies for the cohort.

Cohorts/Characterization/Incidence Rate are WebApi categories, where WebApi manages the execution of generations.

Example: We may want to know if a certain cohort specification has been generated by checking cohort generation status `getCohortGenerationInformation(baseUrl = baseUrl, cohortId= 4234)`. If a cohort is not previously generated, it may be generated using `invokeCohortSetGeneration(baseUrl = baseUrl,`

`cohortId = 4234, sourceKey = 'HCUP')` . If it is already generated, we can extract its output of cohort generation using `getCohortResults(baseUrl, cohortId = 4234)`.

5.2 Characterization

Please review 'Characterization - The Book of OHDSI'.